

Telecom AI Incident Agent

Detailed Project Handbook - Versions 1 to 7

AI Engineering + Data Engineering + Docker + Azure Cloud Deployment

Purpose: This handbook documents exactly what was built, why each tool was used, the code patterns implemented, key commands executed, errors encountered, and how the project evolved from a local FastAPI prototype into an Azure-hosted cloud application.

Project Type	Telecom AI Incident Management Platform
Primary Stack	Python, FastAPI, LangGraph, PostgreSQL, Redis, Kafka, Docker, Azure
Deployment Target	Azure Web App for Containers using Azure Container Registry
Live Health URL	https://telecom-ai-agent-4383.azurewebsites.net/health
Live Swagger URL	https://telecom-ai-agent-4383.azurewebsites.net/docs

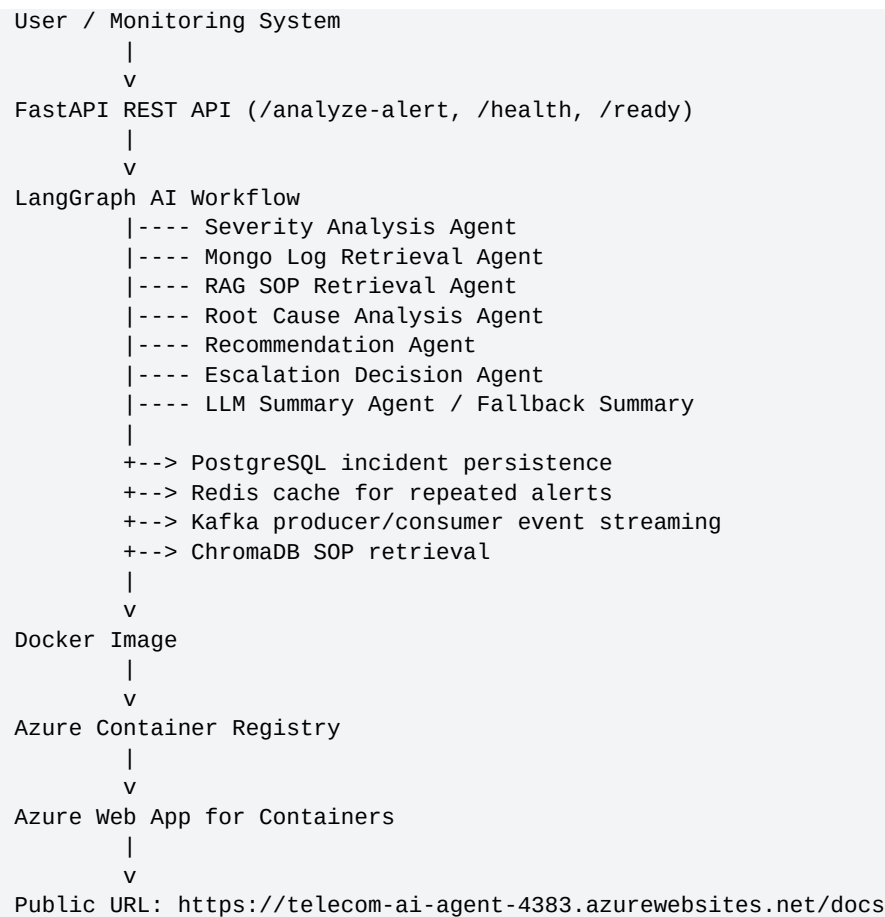
1. Executive Summary

The Telecom AI Incident Agent project is an end-to-end AI platform that simulates how a telecom company can receive tower/network alerts, retrieve related logs, classify severity, identify root cause, recommend operational action, decide escalation, generate an AI-style summary, save the incident, cache repeat responses, process event streams, and finally deploy the API to Azure cloud.

The project is valuable because it combines several real enterprise engineering areas: backend APIs, multi-agent AI workflow, databases, caching, event streaming, data engineering, Docker, cloud deployment, and production troubleshooting.

2. Final Architecture

High-level architecture



3. Tools and Why We Used Them

Tool	Purpose in Project
Python	Main programming language used for API, workflow, data processing, Kafka scripts, and utility

	scripts.
FastAPI	Created REST APIs for alert analysis, incident lookup, health checks, and readiness checks.
Pydantic	Validated incoming API request body fields such as tower_id, issue, packet_loss, and latency_ms.
LangGraph	Orchestrated the incident analysis workflow as a graph of multiple agents/nodes.
PostgreSQL	Stored analyzed incidents permanently so they can be retrieved later.
SQLAlchemy	Connected Python code to PostgreSQL using ORM models and sessions.
MongoDB Atlas	Stored or retrieved telecom logs by tower_id for analysis.
ChromaDB	Used as a vector database for retrieving relevant SOP guidance.
Redis	Cached repeated incident analysis results to improve performance.
Kafka	Simulated real-time telecom alert streaming.
Docker	Packaged the FastAPI app into a portable container.
Docker Compose	Ran multiple local services together: FastAPI, Redis, Kafka, and Zookeeper.
PySpark	Processed larger telecom datasets for analytics and data engineering.
Databricks	Ran Spark processing in a cloud-like notebook environment and created Bronze/Silver/Gold tables.
Azure CLI	Created and managed Azure cloud resources using commands.
Azure Container Registry	Stored the Docker image in Azure cloud.
Azure App Service	Hosted the Dockerized FastAPI application with a public URL.
Git/GitHub	Managed project versions and branches from version 1 through version 7.

4. Version Roadmap Summary

Version	Main Outcome
Version 1	FastAPI + LangGraph incident workflow
Version 2	PostgreSQL persistence and incident APIs
Version 3	Redis caching
Version 4	Kafka event streaming
Version 5	Data engineering with Hugging Face, synthetic data, PostgreSQL export, PySpark, Databricks
Version 6	Dockerization and local container orchestration
Version 7	Azure cloud integration and production deployment

5. Version 1 - FastAPI and LangGraph Workflow

Goal: Build a minimal API that accepts telecom alert details and routes them through an AI-style workflow. This created the foundation of the entire project.

5.1 Main API Code

app/main.py - initial API pattern

```
from fastapi import FastAPI
from pydantic import BaseModel
from app.graph import build_incident_graph

app = FastAPI(title="Minimal Telecom AI Incident Agent")
incident_graph = build_incident_graph()

class AlertRequest(BaseModel):
    tower_id: str
    issue: str
    packet_loss: float
    latency_ms: float

@app.get("/")
def home():
    return {"message": "Minimal Telecom AI Incident Agent is running with LangGraph"}

@app.post("/analyze-alert")
def analyze_alert(alert: AlertRequest):
    result = incident_graph.invoke({
        "tower_id": alert.tower_id,
        "issue": alert.issue,
        "packet_loss": alert.packet_loss,
        "latency_ms": alert.latency_ms,
        "logs": [],
        "severity": "",
        "root_cause": "",
    })
```

```

        "recommendation": "",
        "escalation_required": False,
        "escalation_team": "",
        "sop_guidelines": "",
        "ai_summary": "",
        "agents_executed": []
    })
    return result

```

Explanation: FastAPI exposes REST endpoints. The AlertRequest class validates the request body. The /analyze-alert endpoint sends the alert into the LangGraph workflow and returns the final analysis output.

5.2 LangGraph State and Nodes

app/graph.py - state and example nodes

```

from typing import TypedDict, List
from langgraph.graph import StateGraph, END

class IncidentState(TypedDict):
    tower_id: str
    issue: str
    packet_loss: float
    latency_ms: float
    logs: List[str]
    severity: str
    root_cause: str
    recommendation: str
    escalation_required: bool
    escalation_team: str
    sop_guidelines: str
    ai_summary: str
    agents_executed: List[str]

def analyze_severity_node(state: IncidentState):
    if state["packet_loss"] > 10 and state["latency_ms"] > 250:
        severity = "critical"
    elif state["packet_loss"] > 5:
        severity = "medium"
    else:
        severity = "low"
    return {"severity": severity}

def root_cause_node(state: IncidentState):
    logs_text = " ".join(state["logs"]).lower()
    if "backhaul" in logs_text:
        root_cause = "Possible backhaul instability or fiber provider issue"
    elif "power" in logs_text:
        root_cause = "Possible power issue at tower site"
    elif "latency" in logs_text:
        root_cause = "Possible congestion or routing issue"
    else:
        root_cause = "Root cause unclear. More logs are needed."
    return {"root_cause": root_cause}

```

Explanation: IncidentState defines the shared data structure that flows through the graph. Each node updates part of the state. This makes the workflow modular, easy to debug, and similar to a multi-agent system.

5.3 Graph Routing Logic

LangGraph workflow build

```
def route_escalation(state: IncidentState):
    if state["severity"] == "critical":
        return "escalate"
    return "no_escalation"

def build_incident_graph():
    graph = StateGraph(IncidentState)

    graph.add_node("analyze_severity", analyze_severity_node)
    graph.add_node("retrieve_logs", retrieve_logs_node)
    graph.add_node("read_sop", read_sop_node)
    graph.add_node("root_cause", root_cause_node)
    graph.add_node("recommendation", recommendation_node)
    graph.add_node("escalation", escalation_node)
    graph.add_node("no_escalation", no_escalation_node)
    graph.add_node("summary", summary_node)

    graph.set_entry_point("analyze_severity")
    graph.add_edge("analyze_severity", "retrieve_logs")
    graph.add_edge("retrieve_logs", "read_sop")
    graph.add_edge("read_sop", "root_cause")
    graph.add_edge("root_cause", "recommendation")

    graph.add_conditional_edges(
        "recommendation",
        route_escalation,
        {"escalate": "escalation", "no_escalation": "no_escalation"}
    )

    graph.add_edge("escalation", "summary")
    graph.add_edge("no_escalation", "summary")
    graph.add_edge("summary", END)

    return graph.compile()
```

Explanation: The graph connects the incident analysis stages in order. Conditional routing decides whether an incident should go to escalation or no-escalation branch. This is the core AI workflow.

5.4 Local Test Command

```
uvicorn app.main:app --reload

# Browser:
http://127.0.0.1:8000/docs

# Example request body:
{
    "tower_id": "TX-DAL-102",
    "issue": "High packet loss and latency spike",
```

```
"packet_loss": 18,  
"latency_ms": 340  
}
```

6. Version 2 - PostgreSQL Persistence

Goal: Save every analyzed incident into PostgreSQL so the system has permanent records. This moved the app from stateless API to persistent backend service.

6.1 Database Connection

app/database.py

```
from sqlalchemy import create_engine  
from sqlalchemy.orm import sessionmaker, declarative_base  
import os  
  
DATABASE_URL = os.getenv(  
    "DATABASE_URL",  
    "postgresql://postgres:password@localhost:5432/telecom_ai_db"  
)  
  
engine = create_engine(DATABASE_URL)  
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)  
Base = declarative_base()  
  
try:  
    Base.metadata.create_all(bind=engine)  
    print("Database connected successfully")  
except Exception as e:  
    print(f"Database connection skipped: {e}")
```

Explanation: SQLAlchemy creates a database engine and session factory. The try/except was later added for Azure deployment so the app does not crash if cloud PostgreSQL is not configured yet.

6.2 Incident Model

app/models.py

```
from sqlalchemy import Column, Integer, String, Float, Boolean, DateTime  
from datetime import datetime  
from app.database import Base  
  
class Incident(Base):  
    __tablename__ = "incidents"  
  
    id = Column(Integer, primary_key=True, index=True)  
    tower_id = Column(String)  
    issue = Column(String)  
    packet_loss = Column(Float)  
    latency_ms = Column(Float)  
    severity = Column(String)  
    root_cause = Column(String)  
    recommendation = Column(String)  
    escalation_required = Column(Boolean)
```

```

escalation_team = Column(String)
ai_summary = Column(String)
created_at = Column(DateTime, default=datetime.utcnow)

```

Explanation: This class maps Python fields to a PostgreSQL table. Each analyzed incident becomes one row.

6.3 CRUD Functions

app/crud.py

```

from app.models import Incident

def save_incident(db, result):
    incident = Incident(
        tower_id=result["tower_id"],
        issue=result["issue"],
        packet_loss=result["packet_loss"],
        latency_ms=result["latency_ms"],
        severity=result["severity"],
        root_cause=result["root_cause"],
        recommendation=result["recommendation"],
        escalation_required=result["escalation_required"],
        escalation_team=result["escalation_team"],
        ai_summary=result["ai_summary"]
    )
    db.add(incident)
    db.commit()
    db.refresh(incident)
    return incident

def get_all_incidents(db):
    return db.query(Incident).all()

def get_incident_by_id(db, incident_id: int):
    return db.query(Incident).filter(Incident.id == incident_id).first()

```

6.4 API Endpoints for Stored Incidents

```

@app.get("/incidents")
def get_incidents():
    db = SessionLocal()
    try:
        incidents = get_all_incidents(db)
        return incidents
    finally:
        db.close()

@app.get("/incidents/{incident_id}")
def get_incident(incident_id: int):
    db = SessionLocal()
    try:
        incident = get_incident_by_id(db, incident_id)
        if incident is None:
            raise HTTPException(status_code=404, detail="Incident not found")
        return incident
    finally:
        db.close()

```


Explanation: These endpoints allow users to retrieve all incidents or a specific incident by ID. This is useful for dashboards, audit trails, and operations review.

7. Version 3 - Redis Caching

Goal: Add caching so if the same tower and issue are analyzed again, the API can return the previous result quickly without re-running the workflow.

app/redis_cache.py

```
import redis
import os

REDIS_HOST = os.getenv("REDIS_HOST", "localhost")
REDIS_PORT = int(os.getenv("REDIS_PORT", "6379"))

redis_client = redis.Redis(
    host=REDIS_HOST,
    port=REDIS_PORT,
    decode_responses=True
)
```

Redis cache logic in /analyze-alert

```
cache_key = f"{alert.tower_id}:{alert.issue}"
cached_result = redis_client.get(cache_key)

if cached_result:
    logger.info("Returning result from Redis cache")
    return json.loads(cached_result)

# After workflow and database save
redis_client.set(cache_key, json.dumps(result), ex=300)
```

Explanation: Redis stores a temporary copy of the response for 300 seconds. This reduces repeated work and demonstrates production performance optimization.

Important Docker networking lesson: locally Redis host can be localhost, but inside Docker Compose the FastAPI container should connect to Redis using service name redis. In Azure App Service, Redis is not available unless Azure Cache for Redis is created separately.

8. Version 4 - Kafka Event Streaming

Goal: Simulate telecom monitoring systems sending alerts as event streams. Kafka was used to represent real-time alert ingestion.

8.1 Kafka Producer

app/kafka_producer.py

```

from kafka import KafkaProducer
import json
import time

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

alerts = [
    {"tower_id": "TX-DAL-102", "issue": "High packet loss and latency spike", "packet_loss": 18,
    "latency_ms": 340},
    {"tower_id": "TX-AUS-220", "issue": "Power backup warning", "packet_loss": 2, "latency_ms":
    90},
    {"tower_id": "TX-HOU-310", "issue": "Minor latency fluctuation", "packet_loss": 3,
    "latency_ms": 140}
]

for alert in alerts:
    producer.send("telecom-alerts", alert)
    print(f"Alert sent: {alert}")
    time.sleep(2)

producer.flush()

```

8.2 Kafka Consumer

app/kafka_consumer.py

```

from kafka import KafkaConsumer
import json
import requests

consumer = KafkaConsumer(
    "telecom-alerts",
    bootstrap_servers="localhost:9092",
    value_deserializer=lambda m: json.loads(m.decode("utf-8")),
    auto_offset_reset="earliest",
    group_id="telecom-ai-group"
)

print("Listening for telecom alerts...")

for message in consumer:
    alert = message.value
    print("Received alert:")
    print(alert)

    try:
        response = requests.post("http://127.0.0.1:8000/analyze-alert", json=alert)
        print("Status Code:", response.status_code)
        print("AI Agent Response:", response.json())
    except Exception as e:
        print(f"Error calling AI agent API: {e}")

```

Explanation: Producer sends alerts into Kafka. Consumer receives them and calls the FastAPI AI endpoint. This demonstrates an event-driven telecom alert pipeline.

8.3 Kafka Docker Compose Concept

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    container_name: telecom-zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    container_name: telecom-kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: telecom-zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

9. Version 5 - Data Engineering Layer

Goal: Add data engineering workflows using external dataset ingestion, synthetic data generation, PostgreSQL export, PySpark processing, and Databricks Bronze/Silver/Gold architecture.

9.1 Hugging Face Dataset Ingestion

app/hf_dataset_loader.py

```
from datasets import load_dataset
import os

output_dir = "data/huggingface"
os.makedirs(output_dir, exist_ok=True)

dataset = load_dataset("aai510-group1/telco-customer-churn", split="train")
df = dataset.to_pandas()

output_path = os.path.join(output_dir, "telco_customer_churn.csv")
df.to_csv(output_path, index=False)

print(f"Hugging Face dataset saved to {output_path}")
print(df.head())
```

Explanation: This introduced external data ingestion from Hugging Face. It shows how enterprise AI/data projects pull datasets from external sources.

9.2 Synthetic Telecom Dataset

app/generate_synthetic_data.py

```
import random
import os
from datetime import datetime, timedelta
import pandas as pd

output_dir = "data/synthetic"
os.makedirs(output_dir, exist_ok=True)

tower_ids = ["TX-DAL-102", "TX-AUS-220", "TX-HOU-310", "TX-SAN-410", "TX-ELP-510", "TX-FRW-610"]
regions = {
    "TX-DAL-102": "Dallas",
    "TX-AUS-220": "Austin",
    "TX-HOU-310": "Houston",
    "TX-SAN-410": "SanAntonio",
    "TX-ELP-510": "ElPaso",
    "TX-FRW-610": "FortWorth"
}

records = []
start_time = datetime(2026, 6, 1, 0, 0, 0)

for i in range(10000):
    tower_id = random.choice(tower_ids)
    packet_loss = round(random.uniform(0, 25), 2)
    latency_ms = round(random.uniform(50, 500), 2)

    if packet_loss > 10 and latency_ms > 250:
        severity = "critical"
    elif packet_loss > 5:
        severity = "medium"
    else:
        severity = "low"

    records.append({
        "event_id": i + 1,
        "tower_id": tower_id,
        "region": regions[tower_id],
        "packet_loss": packet_loss,
        "latency_ms": latency_ms,
        "severity": severity,
        "timestamp": start_time + timedelta(minutes=i)
    })

pd.DataFrame(records).to_csv("data/synthetic/synthetic_telecom_incidents.csv", index=False)
```

Explanation: Synthetic data lets us simulate thousands of telecom incidents for analytics when real production data is not available.

9.3 PostgreSQL Export

app/export_postgres_incidents.py

```
import os
import pandas as pd
```

```

from app.database import SessionLocal
from app.models import Incident

output_dir = "data/postgres_export"
os.makedirs(output_dir, exist_ok=True)

db = SessionLocal()
try:
    incidents = db.query(Incident).all()
    data = []
    for incident in incidents:
        data.append({
            "id": incident.id,
            "tower_id": incident.tower_id,
            "issue": incident.issue,
            "packet_loss": incident.packet_loss,
            "latency_ms": incident.latency_ms,
            "severity": incident.severity,
            "root_cause": incident.root_cause,
            "recommendation": incident.recommendation,
            "created_at": incident.created_at
        })
    pd.DataFrame(data).to_csv("data/postgres_export/postgres_incidents_export.csv", index=False)
finally:
    db.close()

```

9.4 PySpark Analytics

app/spark_analytics.py

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import count, avg

spark = SparkSession.builder.appName("TelecomDataEngineeringAnalytics").getOrCreate()

synthetic_df = spark.read.csv("data/synthetic/synthetic_telecom_incidents.csv", header=True,
inferSchema=True)

synthetic_df.groupBy("severity").agg(
    count("*").alias("incident_count")
).show()

synthetic_df.groupBy("region").agg(
    avg("latency_ms").alias("avg_latency")
).show()

postgres_df = spark.read.csv("data/postgres_export/postgres_incidents_export.csv", header=True,
inferSchema=True)
postgres_df.groupBy("severity").agg(count("*").alias("incident_count")).show()

hf_df = spark.read.csv("data/huggingface/telco_customer_churn.csv", header=True, inferSchema=True)
hf_df.printSchema()

spark.stop()

```

9.5 Databricks Bronze/Silver/Gold Pipeline

```
from pyspark.sql.functions import col, count, avg

synthetic_path = "/Volumes/workspace/default/telecom_data/synthetic_telecom_incidents.csv"
synthetic_df = spark.read.csv(synthetic_path, header=True, inferSchema=True)

# Bronze: raw data
synthetic_df.write.format("delta").mode("overwrite").saveAsTable("telecom_bronze")

# Silver: cleaned data
silver_df = synthetic_df.filter(
    col("packet_loss").isNotNull() & col("latency_ms").isNotNull()
).dropDuplicates()
silver_df.write.format("delta").mode("overwrite").saveAsTable("telecom_silver")

# Gold: business-ready analytics
gold_df = silver_df.groupBy("region", "severity").agg(
    count("*").alias("incident_count"),
    avg("latency_ms").alias("avg_latency")
)
gold_df.write.format("delta").mode("overwrite").saveAsTable("telecom_gold")

display(gold_df)
```

Explanation: Bronze stores raw records, Silver stores cleaned records, and Gold stores analytics-ready KPIs. This is a common Databricks/Delta Lake architecture used in enterprise data engineering.

10. Version 6 - Dockerization and Local Container Orchestration

Goal: Package the FastAPI application into a Docker image and run supporting services using Docker Compose.

10.1 Dockerfile

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Explanation: Dockerfile defines how to build the app image. It installs dependencies, copies code, exposes port 8000, and starts Uvicorn.

10.2 Docker Compose

docker-compose.yml

```
services:
  telecom-ai-agent:
    build: .
    container_name: telecom-ai-agent
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - redis
      - zookeeper
      - kafka

  redis:
    image: redis
    container_name: telecom-redis
    ports:
      - "6379:6379"

  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    container_name: telecom-zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    container_name: telecom-kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: telecom-zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Explanation: Docker Compose starts multiple containers together. FastAPI, Redis, Kafka, and Zookeeper run as separate services but share a Docker network.

Important concept: Docker Compose did not replace Uvicorn. Uvicorn still runs the FastAPI app, but now it runs inside the Docker container automatically.

10.3 Docker Commands Used

```
docker-compose up --build

docker ps

docker logs telecom-ai-agent
```

```
docker-compose down

docker rm -f telecom-redis
docker rm -f telecom-kafka
docker rm -f telecom-zookeeper
```

11. Version 7 - Production Health, Logging, and Azure Deployment

Goal: Prepare the application for production and deploy it to Azure cloud with a public URL.

11.1 Health and Readiness APIs

Production health endpoints

```
@app.get("/health")
def health_check():
    return {
        "status": "healthy",
        "service": "telecom-ai-agent",
        "version": "7.0"
    }

@app.get("/ready")
def readiness_check():
    return {
        "status": "ready",
        "dependencies": {
            "fastapi": "running",
            "langgraph": "loaded",
            "redis": "configured",
            "kafka": "configured",
            "postgresql": "configured",
            "mongodb": "configured"
        }
    }
```

Explanation: /health confirms the service is alive. /ready confirms the app is configured and ready. Cloud platforms use these endpoints for monitoring and startup checks.

11.2 Structured Logging

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%asctime)s - %(levelname)s - %(message)s"
)

logger = logging.getLogger(__name__)

@app.post("/analyze-alert")
def analyze_alert(alert: AlertRequest):
    logger.info(f"Received alert for tower: {alert.tower_id}")
```



```
# workflow runs here
```

```
logger.info(f"Incident saved with database_id: {saved_incident.id}")
```

Explanation: Production logs help diagnose request flow, database saves, and API behavior inside Docker or Azure logs.

11.3 Azure Files Created

azure.yaml

```
# azure.yaml
name: telecom-ai-agent

metadata:
  template: telecom-ai-agent@0.0.1

infra:
  provider: bicep
  path: infra

services:
  api:
    project: .
    language: docker
    host: containerapp
```

Explanation: azure.yaml tells Azure Developer CLI how the project should be deployed and where infrastructure files are located.

Bicep infrastructure template

```
# infra/main.bicep
targetScope = 'resourceGroup'

param environmentName string
param location string = resourceGroup().location
param appName string = 'telecom-ai-agent'

resource logAnalytics 'Microsoft.OperationalInsights/workspaces@2022-10-01' = {
  name: '${appName}-logs-${environmentName}'
  location: location
  properties: {
    sku: {
      name: 'PerGB2018'
    }
    retentionInDays: 30
  }
}

resource containerAppEnv 'Microsoft.App/managedEnvironments@2023-05-01' = {
  name: '${appName}-env-${environmentName}'
  location: location
  properties: {
    appLogsConfiguration: {
      destination: 'log-analytics'
    }
  }
}
```

```

    logAnalyticsConfiguration: {
      customerId: logAnalytics.properties.customerId
      sharedKey: logAnalytics.listKeys().primarySharedKey
    }
  }
}
}

```

Explanation: Bicep is Infrastructure as Code for Azure. It defines cloud resources such as Log Analytics and Container App Environment. Even though Container Apps was later restricted by subscription policy, this gave hands-on IaC experience.

Production startup script

```

# start.sh
#!/bin/sh

uvicorn app.main:app --host 0.0.0.0 --port 8000

```

11.4 Azure CLI Deployment Commands

Azure command sequence

```

# Login
az login
azd auth login --use-device-code

# Create resource group
az group create --name telecom-ai-rg --location eastus

# Register Container Registry provider
az provider register --namespace Microsoft.ContainerRegistry
az provider show --namespace Microsoft.ContainerRegistry --query "registrationState" -o table

# Create Azure Container Registry
az acr create --resource-group telecom-ai-rg --name telecomaiacr4383 --sku Basic --location eastus2

# Login to registry
az acr login --name telecomaiacr4383

# Tag local image into Azure registry format
docker tag telecom-ai-agent-v1-telecom-ai-agent:latest telecomaiacr4383.azurecr.io/telecom-ai-agent:latest

# Push image to Azure Container Registry
docker push telecomaiacr4383.azurecr.io/telecom-ai-agent:latest

# Create App Service Plan
az appservice plan create --name telecom-ai-plan --resource-group telecom-ai-rg --sku B1 --is-linux --location eastus2

# Create Web App from container image
az webapp create --resource-group telecom-ai-rg --plan telecom-ai-plan --name telecom-ai-agent-4383 --deployment-container-image-name telecomaiacr4383.azurecr.io/telecom-ai-agent:latest

```

```
# Enable ACR admin access
az acr update -n telecomaiacr4383 --admin-enabled true

# Configure App Service to use container image
az webapp config container set --name telecom-ai-agent-4383 --resource-group telecom-ai-rg --
docker-custom-image-name telecomaiacr4383.azurecr.io/telecom-ai-agent:latest --docker-registry-
server-url https://telecomaiacr4383.azurecr.io --docker-registry-server-user telecomaiacr4383 --
docker-registry-server-password "<ACR_PASSWORD>"

# Set FastAPI container port
az webapp config appsettings set --resource-group telecom-ai-rg --name telecom-ai-agent-4383 --
settings WEBSITES_PORT=8000

# Restart web app
az webapp restart --name telecom-ai-agent-4383 --resource-group telecom-ai-rg
```

Security note: Do not commit real API keys, MongoDB URI values, Azure passwords, or ACR passwords to GitHub. Use environment variables or Azure Key Vault. Rotate any credential that was exposed in terminal screenshots or chat.

11.5 Final Azure URLs

- Swagger UI: <https://telecom-ai-agent-4383.azurewebsites.net/docs>
- Health endpoint: <https://telecom-ai-agent-4383.azurewebsites.net/health>
- Azure Web App: telecom-ai-agent-4383
- Azure Resource Group: telecom-ai-rg
- Azure Container Registry: telecomaiacr4383.azurecr.io
- Azure App Service Plan: telecom-ai-plan

12. Troubleshooting Log and Fixes

Issue	Fix / Learning
ModuleNotFoundError: app.llm_gateway	File naming/import issue. Fixed by matching filename and import path.
Fallback LLM summary returned	LLM service unavailable. Added fallback summary so workflow still succeeds.
Docker container name conflict	Old containers such as telecom-redis and telecom-zookeeper already existed. Fixed using docker rm -f and docker-compose down.
Redis localhost connection refused inside Docker	Inside Docker localhost means the FastAPI container itself. Fixed by using service name redis or environment variable REDIS_HOST.
PostgreSQL host.docker.internal works locally but not Azure	host.docker.internal exists in local Docker Desktop but not Azure cloud. Fixed by making DB startup

	optional and planning future Azure PostgreSQL.
Azure Container Apps region blocked	Azure for Students subscription restricted Microsoft.App/managedEnvironments. Switched to Azure App Service for Containers.
Microsoft.ContainerRegistry missing provider	Registered provider using az provider register --namespace Microsoft.ContainerRegistry.
Docker Desktop engine failed	Restarted Docker/WSL/laptop until docker ps worked.
Azure container startup timeout	Checked Azure logs using az webapp log tail, identified PostgreSQL hostname crash, fixed startup behavior.
ACR credential issue	Enabled ACR admin access and configured Web App registry credentials.

13. Localhost vs Azure Deployment

Area	Localhost	Azure
Where it runs	Your laptop	Microsoft cloud data center
URL	localhost or 127.0.0.1	azurewebsites.net public URL
Availability	Only while laptop/server is running	Hosted by Azure
Access	Mostly only you	Public internet access
Purpose	Development/testing	Production/cloud deployment
Infrastructure	Manual local services	Cloud services and managed hosting

14. Git/GitHub Version Control Flow

```
# Create version branch
git checkout main
git pull origin main
git checkout -b version-7

# Check changes
git status

# Stage files
git add .
```

```
# Commit changes
git commit -m "Version 7 - Azure production deployment completed"

# Push branch
git push origin version-7

# Then create Pull Request in GitHub and merge into main
```

Explanation: Each major project milestone was separated into version branches. This makes the GitHub history easy to understand and professional.

15. Suggested README Section for Version 7

```
## Version 7 - Azure Production Deployment

Version 7 deployed the Telecom AI Incident Agent to Azure using Docker and Azure App Service for Containers.

### Cloud Resources
- Azure Resource Group: telecom-ai-rg
- Azure Container Registry: telecomaiacr4383.azurecr.io
- Azure App Service Plan: telecom-ai-plan
- Azure Web App: telecom-ai-agent-4383

### Live Endpoints
- Swagger UI: https://telecom-ai-agent-4383.azurewebsites.net/docs
- Health Check: https://telecom-ai-agent-4383.azurewebsites.net/health

### Key Features
- Dockerized FastAPI application
- Azure Container Registry image hosting
- Azure Web App container deployment
- Production health and readiness APIs
- Structured logging
- Azure troubleshooting and deployment debugging

### Important Learning
The project showed the difference between local Docker deployment and cloud deployment. Local services such as PostgreSQL and Redis need cloud-managed replacements in real production environments.
```

16. Interview Explanation

Short project explanation:

I built a telecom AI incident management platform using FastAPI, LangGraph, PostgreSQL, Redis, Kafka, Docker, Databricks, and Azure. The system analyzes telecom tower alerts, retrieves logs, classifies severity, identifies root cause, recommends actions, decides escalation, generates summaries, stores incidents, caches repeated requests, and was deployed as a Dockerized API on Azure Web App using Azure Container Registry.

Detailed interview explanation:

The project started as a local FastAPI prototype and evolved into a cloud-native AI platform. I used LangGraph to design the incident workflow as multiple agents, PostgreSQL for persistence, Redis for caching, Kafka for streaming simulation, PySpark and Databricks for data engineering, Docker for containerization, and Azure for cloud deployment. I also handled real deployment issues such as Azure region restrictions, provider registration, Docker engine failures, ACR authentication, container startup failures, and cloud database hostname differences.

17. Next Recommended Versions

- Version 8: Azure PostgreSQL and Azure Cache for Redis integration.
- Version 9: GitHub Actions CI/CD pipeline to build and push Docker images automatically.
- Version 10: Monitoring with Azure Monitor, Application Insights, Prometheus, or Grafana.
- Version 11: Authentication using JWT or OAuth.
- Version 12: Kubernetes/AKS deployment.
- Version 13: Advanced multi-agent AI planning and self-healing workflows.

18. Final Outcome

By Version 7, the project became a cloud-hosted telecom AI incident management platform. It demonstrates AI engineering, backend engineering, data engineering, DevOps, containerization, cloud deployment, and real-world troubleshooting. The project is strong for GitHub portfolio, resume discussion, and AI/cloud engineering interviews.